

STUDY GUIDE · DESK REFERENCE

Git and GitHub

for C.G. · CyClaw operator

cgfixit/CyClaw · Windows + Apple Silicon · Aug 27, 2026

This guide collapses the last three working replies — command catalog, history + 16 workflows, and the Windows clone path — into one printable reference. It is general Git and GitHub first. CyClaw conventions are called out where they change what you type.

Who this is for

Christopher Grady (C.G. / @cgfixit), Atlanta. Daily CyClaw work happens on Windows at %USERPROFILE%\CyClaw\repo and on the M5 Pro Mac at ~/.CyClaw/repo. Flagship repo: <https://github.com/cgfixit/CyClaw>. Agents cut branches as grok/, codex/, or kimi/.

Contents

1. Mental model — snapshots, labels, remotes
2. Brief history of Git and GitHub
3. Your machines and clone paths
4. CyClaw + GitHub conventions
5. Command catalog
6. Sixteen workflows (PowerShell and zsh/bash)
7. Case study — PR #1168 dirty merge
8. GitHub CLI you will actually use
9. Daily loop, anti-patterns, recovery
10. One-page cheat sheet

1. Mental model

Git is not a file-lock server. It is a content-addressed database of **snapshots**. Every commit stores a full tree of files, a pointer to one or more parents, an author, and a message. The SHA is a hash of that object. If the bits change, the name changes.

Piece	What it actually is
blob	File contents. Same bytes = same hash, even across commits.
tree	A directory listing: names → blobs or other trees.
commit	Snapshot + parent pointer(s) + message. History is a DAG.
branch	A movable label on a commit. <code>main</code> is not special to Git.
HEAD	“What I have checked out.” Usually a pointer to a branch label.
remote	A named URL plus a set of labels (<code>origin/main</code>).
index / staging	The next commit, assembled hunk by hunk with <code>git add</code> .
reflog	Local diary of every HEAD move on this machine. Not on GitHub.

The sentence that prevents most damage

A commit already pushed to a shared CyClaw PR is public history. Do not `reset --hard` it. `git revert` adds a new commit that undoes the old one. That is why the #1168 stub commits get reverted, not erased.

2. Brief history

When	What happened
Apr 2005	Linus Torvalds starts Git after BitKeeper's free license is pulled. Goals: speed, integrity, cheap local branches, no required central server.
Name	British slang for a stubborn person. Torvalds applied it to himself and the tool.
Jun 2005	First Linux kernel release managed with Git.
2005–07	Junio Hamano becomes maintainer. Plumbing (objects, refs) stays low-level; porcelain (commit, later switch/restore) is what people type.
2008	GitHub launches. Pull requests replace “email a patch.” Hosting + review beat CVS/SVN/Hg for most teams.
2014–	<code>git worktree</code> . 2019: switch/restore split overloaded checkout. Default branch name shifts to <code>main</code> .
Today	Same object model. GitHub/GitLab/Forgejo are UIs on it. Protocol is migrating SHA-1 → SHA-256; you still see 40-character SHAs everywhere.

What has not changed: a commit is a snapshot plus parent pointer(s). Branches are movable labels. Remotes are another set of labels (`origin/main`).

3. Your machines and clone paths

CyClaw's installer default is a hidden home directory plus a repo folder inside it. Home holds `.env`, `venv` (Mac), and operator state. Git lives under `repo`.

Machine	Clone path	Shell
Windows daily box	%USERPROFILE%\CyClaw\repo expands to C:\Users\<you>\CyClaw\repo	PowerShell 7. Git for Windows on PATH.
M5 Pro Mac (48 GB, Tahoe 26.6, arm64)	~/.CyClaw/repo	zsh login shell. Homebrew at /opt/homebrew. Git via brew or Xcode CLT.
This Grok sandbox	No local clone. Remote only.	Do the revert/merge on your machine, not here.

Enter the repo

Windows PowerShell

```
cd $env:USERPROFILE\CyClaw\repo
git status -sb
git remote -v
```

Default Windows clone. If missing, you passed -RepoPath at install.

macOS zsh

```
cd ~/.CyClaw/repo
git status -sb
git remote -v
# confirm arm64 git, not a Rosetta copy
file $(command -v git)
```

Apple Silicon: prefer /opt/homebrew/bin/git or /usr/bin/git. Avoid sudo brew.

Find a non-default clone

Windows

```
Get-ChildItem -Path $env:USERPROFILE, $env:USERPROFILE\source,
  $env:USERPROFILE\Documents -Directory -Filter CyClaw -Recurse
  -ErrorAction SilentlyContinue |
Where-Object { Test-Path (Join-Path $_.FullName '.git') } |
Select-Object -ExpandProperty FullName
```

macOS zsh

```
find ~ ~/src ~/Documents ~/.CyClaw -maxdepth 3 -type d -name CyClaw \
  2>/dev/null | while read -r d; do
  [[ -d "$d/.git" ]] && echo "$d"
done
```

4. CyClaw + GitHub conventions

These are project rules, not Git rules. Breaking them fails hook or advisory CI even when Git is happy. Source of truth for PR bodies: .github/PULL_REQUEST_TEMPLATE.md.

Rule	What you type
Remote / owner	https://github.com/cgfixit/CyClaw — owner casing is cgfixit
Default branch	main (base of every PR unless you are stacking)
Grok Build branches	grok/<kebab-case-feature> example: grok/netconnect-docs
Other agent prefixes	codex/ and kimi/ are allowlisted. #1168 is codex/netconnect-passive. #1170 is kimi/scan-conda-torch.

Rule	What you type
Commit / PR title	[prefix] - Short sentence. Prefixes: [invariant] [governance] [fsconnect] [agentic] [rag] [harness] [security] [docs] [infra] [fix] [feat]
PR body	Fill every template section. Proposed changes, invariant impact, types, benefits, risks, checklist, further comments. No free-form substitute.
Local gates	.githubhooks/ (branch name + subject prefix), scripts/check-pr-template.sh before gh pr create, advisory pr-template-check.yml
Invariants	I1 RAG-first · I2 topology=policy · I3 external fallback gated · I4 audit convergence · I5 soul human-gated · I6 module isolation. Do not import OOB packages into the request path.

Secrets never belong in Git

CYCLAW_API_KEY, GROK_API_KEY, ANTHROPIC_API_KEY, GH_TOKEN live in Keychain / CredMan / owner-only dotenv. Windows: %USERPROFILE%\ .CyClaw\ .env with an ACL that only you can read (icacls). Mac: ~/ .CyClaw/ .env mode 600 or 400. Invoke scripts refuse group/world-readable dotenv. Never commit .env, never inline a key in a LaunchAgent plist.

5. Command catalog

Same commands on Windows and Mac. Only paths and line-continuation differ (PowerShell backtick vs zsh backslash).

Setup

Command	What it does
git config --global user.name "Name"	Author name on every commit from this machine.
git config --global user.email "you@..."	Must match the GitHub account you want attributed.
git config --global init.defaultBranch main	New repos start on main.
git config --global core.autocrlf true	Windows only: LF↔CRLF so you do not fight Linux/macOS.
git config --global core.autocrlf input	Mac/Linux: store LF, leave working tree alone.
git config --global core.editor "code --wait"	VS Code for commit messages and rebase todos.
git config --list	Every setting in force (system → global → local).
git config --show-origin core.autocrlf	Which file set that value.

Start a repo

Command	What it does
git init	Turn this folder into a repo. Creates .git/.
git clone <url>	Copy remote history + default branch into a new folder.
git clone <url> <dir>	Same, name the folder.
git clone --branch codex/netconnect-passive <url>	Clone and check out that branch immediately.
git clone --depth 1 <url>	Shallow: latest commit only. Faster, worse history.

Look around

Command	What it does
<code>git status</code>	Working tree vs index vs HEAD. First command when lost.
<code>git status -sb</code>	Short branch + dirty-file list.
<code>git log --oneline --decorate --graph --all</code>	Compact history of every branch with merge lines.
<code>git log -5</code>	Last five commits.
<code>git show</code>	Latest commit message + full diff.
<code>git show 3f77eca</code>	That commit's message + diff. Last good setup-guide on #1168.
<code>git diff</code>	Unstaged (working tree vs index).
<code>git diff --staged</code>	Staged (index vs HEAD). What the next commit contains.
<code>git diff main...HEAD</code>	Everything this branch changed since it diverged from main.
<code>git blame setup-guide.md</code>	Line-by-line last-touch author/commit.

Branches

Command	What it does
<code>git branch -vv</code>	Local branches and the remote each tracks.
<code>git branch -a</code>	Local + remote-tracking (origin/...).
<code>git switch <branch></code>	Switch branches. Prefer this over checkout for this job.
<code>git switch -c grok/foo</code>	Create grok/foo from HEAD and switch to it.
<code>git branch -d grok/foo</code>	Delete a merged local branch. Refuses if unmerged.
<code>git branch -D grok/foo</code>	Force-delete an unmerged local branch.
<code>git branch -m old new</code>	Rename current/local branch.

Stage and commit

Command	What it does
<code>git add -p</code>	Stage hunk-by-hunk. Use this on mixed files instead of add .
<code>git add <file></code>	Stage that file.
<code>git restore --staged <file></code>	Unstage. File stays modified on disk.
<code>git restore <file></code>	Throw away unstaged edits. Destructive.
<code>git commit -m "[docs] - ..."</code>	Commit with CyClaw-prefixed subject.
<code>git commit --amend</code>	Rewrite last local commit. Do not amend already-pushed commits.
<code>git commit --no-edit</code>	Finish a merge using the existing message.
<code>git rm <file></code>	Delete from disk and stage the deletion.
<code>git mv old new</code>	Rename and stage as a rename.

Undo — pick the right one

Command	What it does
<code>git reset --soft HEAD~1</code>	Drop last commit; keep changes staged. Local only.
<code>git reset --mixed HEAD~1</code>	Drop last commit; keep changes unstaged.

Command	What it does
<code>git reset --hard HEAD~1</code>	Drop last commit and delete its changes. Reflog is the undo.
<code>git revert <sha></code>	New commit that undoes sha. Safe on shared PR branches.
<code>git revert --no-edit 182b8358 51e26f76</code>	#1168: undo the two stub commits, newest first.
<code>git clean -n</code>	Preview untracked files that would be deleted.
<code>git clean -fd</code>	Delete untracked files and folders.
<code>git reflog</code>	Every HEAD move on this machine. Recovery map.
<code>git reset --hard ORIG_HEAD</code>	Jump back to where HEAD was before the last merge/rebase/reset.

Remotes and sync

Command	What it does
<code>git remote -v</code>	Remotes and URLs.
<code>git fetch origin</code>	Download commits/branches. Does not change your files.
<code>git fetch --all --prune</code>	Fetch every remote; drop stale origin/old-branch refs.
<code>git pull --rebase origin main</code>	Fetch, then replay your local commits on latest main.
<code>git pull --ff-only</code>	Fast-forward only. Refuses if a merge would be required.
<code>git push -u origin HEAD</code>	Upload current branch under the same name; set upstream.
<code>git push origin --delete <branch></code>	Delete a remote branch.
<code>git push --force-with-lease</code>	Overwrite remote only if nobody else pushed since your fetch.
<code>git push --force</code>	Overwrite no matter what. Last resort.

Merge, rebase, conflicts

Command	What it does
<code>git merge origin/main</code>	Bring main into this branch. Merge commit if histories diverged.
<code>git merge --abort</code>	Cancel an in-progress merge.
<code>git rebase origin/main</code>	Replay this branch onto latest main. Rewrites SHAs.
<code>git rebase --continue</code>	After you fix a rebase conflict.
<code>git rebase --abort</code>	Cancel an in-progress rebase.
<code>git rebase -i HEAD~3</code>	Interactive rebase of last 3 (squash, reword, drop, reorder).
<code>git cherry-pick <sha></code>	Copy one commit onto the current branch.
<code>git add <file></code>	Mark a conflict resolved after you edit the file.
<code>git diff --check</code>	Flag leftover <<<<<< markers.

Stash, tags, ignore

Command	What it does
<code>git stash push -u -m "wip"</code>	Shelve tracked + untracked work.
<code>git stash list / pop / apply / drop</code>	List, restore-and-drop, restore-keep, delete one.
<code>git tag -a v1.9.1 -m "..."</code>	Annotated release tag on HEAD.

Command	What it does
<code>git push origin v1.9.1</code>	Publish that tag.
<code>git check-ignore -v <file></code>	Why that file is ignored.
<code>git rm --cached <file></code>	Stop tracking a now-ignored file. Leaves it on disk.

Compare and hunt

Command	What it does
<code>git log origin/main...HEAD</code>	Commits on this branch not in main.
<code>git log HEAD...origin/main</code>	Commits on main you do not have yet.
<code>git diff origin/main...HEAD</code>	PR-shaped diff (changes since merge-base).
<code>git merge-base main HEAD</code>	Common ancestor. Useful before a merge.
<code>git bisect start</code>	Binary search for the commit that introduced a bug.
<code>git shortlog -sn</code>	Commit counts by author.

6. Sixteen workflows

Each workflow is shown twice when the commands differ. On the Mac, default shell is zsh; these snippets are POSIX-safe bash/zsh.

6.1 First clone and daily update

Windows

```
git clone https://github.com/cgfixit/CyClaw.git
cd CyClaw
# or the installer clone:
cd %env:USERPROFILE%\CyClaw\repo
git status -sb
git fetch origin
git pull --rebase origin main
```

macOS

```
git clone https://github.com/cgfixit/CyClaw.git
cd CyClaw
# or:
cd ~/.CyClaw/repo
git status -sb
git fetch origin
git pull --rebase origin main
```

`clone` copies history and checks out main. `fetch` downloads. `pull --rebase` replays any local commits on latest main instead of making a “Merge branch main” commit.

6.2 Start a feature branch, commit, push, open a PR

```
git fetch origin
git switch main
git pull --rebase origin main
git switch -c grok/netconnect-docs

# edit files...
git status -sb
git add -p setup-guide.md
git diff --staged
git commit -m "[docs] - keep fsconnect largest and netconnect sections"
git push -u origin grok/netconnect-docs

gh pr create --base main --head grok/netconnect-docs \
  --title "[docs] - keep fsconnect largest and netconnect sections"
```

Then paste the full CyClaw PR template body. Agents still have to fill every section even though `pr-template-check.yml` is advisory.

6.3 See what you changed before committing

```
git status
git diff                # unstaged
git diff --staged       # what commit will record
git log --oneline -8
git log main..HEAD      # commits on this branch not in main
```

6.4 Undo the last commit (not pushed)

```
git reset --soft HEAD~1  # commit gone, changes still staged
# or
git commit --amend       # keep commit, fix message or add a file
```

If it **was** pushed, do not reset. Use `git revert HEAD` so history stays honest.

6.5 Undo a bad pushed commit (the #1168 stub case)

Two stub commits landed on `codex/netconnect-passive`: 51e26f76, then 182b8358. Revert newest first.

Windows

```
cd $env:USERPROFILE\.CyClaw\repo
git fetch origin
git switch codex/netconnect-passive
git pull --ff-only origin codex/netconnect-passive
git revert --no-edit 182b8358 51e26f76
git push origin codex/netconnect-passive
```

macOS

```
cd ~/.CyClaw/repo
git fetch origin
git switch codex/netconnect-passive
git pull --ff-only origin codex/netconnect-passive
git revert --no-edit 182b8358 51e26f76
git push origin codex/netconnect-passive
```

`revert` adds new commits that invert those diffs. Safe on a shared PR branch. `reset --hard` would rewrite history and need a force-push.

6.6 Update your branch with main (merge)


```
git fetch origin
git switch codex/netconnect-passive
git merge origin/main
# if conflict: edit setup-guide.md – keep BOTH sections
# delete <<<<<< ===== >>>>>> markers
git add setup-guide.md
git commit --no-edit
git push origin codex/netconnect-passive
```

Use merge when the PR already has review history and you do not want rewritten SHAs.

6.7 Update your branch with main (rebase, linear)

```
git fetch origin
git switch grok/netconnect-docs
git rebase origin/main
# on conflict: edit file, git add setup-guide.md
git rebase --continue
git push --force-with-lease origin grok/netconnect-docs
```

Rebase rewrites your branch commits. `--force-with-lease` refuses to overwrite if someone else pushed in the meantime. Abort either path with `git merge --abort` or `git rebase --abort`.

6.8 Resolve a conflict (one file)

Git stops and marks the file. For #1168 the hunk is in the “Beyond the core RAG gateway” block of `setup-guide.md`.

```
<<<<<< HEAD
netconnect how-to from this branch
=====
fsconnect largest how-to from main
>>>>>> origin/main
```

Keep both blocks. `fsconnect` largest first (from main / #1167), then the passive `netconnect` how-to (from this PR). Delete the marker lines.

```
git add setup-guide.md
git status          # should say all conflicts fixed
git diff --check    # leftover <<<<<< is a fail
git commit --no-edit # merge
# or: git rebase --continue
```

6.9 Park unfinished work and switch branches

```
git stash push -u -m "wip windows scope filter"
git switch main
# ... hotfix ...
git switch grok/netconnect-docs
git stash list
git stash pop
```

If pop conflicts, fix, `git add`, then `git stash drop` if the stash is still listed.

6.10 Hotfix on main without losing feature work

```
git stash push -m "feature in progress"
git switch main
git pull --rebase origin main
git switch -c grok/fix-otel-t13
# fix, commit, push, PR
git switch grok/netconnect-docs
git stash pop
```

6.11 Copy one commit onto another branch

```
git log --oneline origin/main..codex/netconnect-passive
git switch grok/docs-only
git cherry-pick 3f77eca
git push -u origin grok/docs-only
```

Use when you want the Windows ARP _scope_interface filter without the rest of the PR.

6.12 Delete a merged branch

```
git switch main
git pull --rebase origin main
git branch -d grok/fix-otel-t13
git push origin --delete grok/fix-otel-t13
git fetch --prune
```

-d refuses if Git does not think it is merged. -D force-deletes locally.

6.13 Recover “I reset the wrong thing”

```
git reflog
# find the line before the mistake, e.g. HEAD@{3}
git switch -c recovery HEAD@{3}
# or move current branch back:
git reset --hard HEAD@{3}
```

Reflog is local only. It does not live on GitHub. It is why a bad reset --hard is usually recoverable for ~90 days on the machine that did it.

6.14 Compare this PR to main before you push

```
git fetch origin
git log --oneline origin/main...HEAD
git diff origin/main...HEAD
git diff --stat origin/main...HEAD
```

Triple-dot A . . B = what this branch changed since the merge-base, which matches a GitHub PR diff.

6.15 Ship a tagged release

```
git switch main
git pull --ff-only origin main
git tag -a v1.9.1 -m "v1.9.1 — netconnect passive + conda torch pin"
git push origin main
git push origin v1.9.1
```

6.16 Ignore a file you already committed

```
# add path to .gitignore first
git rm --cached secrets.env
git add .gitignore
git commit -m "[chore] - stop tracking secrets.env"
```

--cached untracks; the file stays on disk.

7. Case study — PR #1168 dirty merge

This is the incident the last three replies were solving. Keep it as the worked example of “docs conflict + bad Contents-API stub + how to recover without rewriting shared history.”

Fact	Value
PR	#1168 [agentic] - add passive scoped network connector
URL	https://github.com/cgfixit/CyClaw/pull/1168
Branch	codex/netconnect-passive
Base	main @ ea8f8035 (at the time of the dirty state)
mergeable_state	dirty — GitHub will not auto-merge or “Update branch”
Real conflict file	setup-guide.md only
Clean auto-merges	README.md and config.yaml already have both connectors
Cause	#1167 inserted fsconnect largest into the same “Beyond the core RAG gateway” block this PR used for netconnect

Commit timeline on the PR

SHA	Role
c666cf6	Original netconnect feature. Good.
3f77eca	Windows ARP_scope_interface filter. Last known full setup-guide.md on the branch.
51e26f76	Bad Contents-API stub while dodging the conflict hunk. Do not merge.
182b8358	Second stub: file became SEE_LOCAL_FILE_RESTORE_FAILED_USE_GIT_SHOW_3f77eca. HEAD when dirty.

Correct resolution

1. Revert the two stubs so setup-guide.md returns to 3f77eca. 2. Merge origin/main. 3. In the conflicted file, keep **both** sections: fsconnect largest from #1167, then the passive netconnect how-to from this PR. 4. Commit the merge and push. I6, passive-only posture, and no /ops route stay unchanged — this is docs only.

Do not merge 51e26f76 or 182b8358 as-is

Those two commits are the failed resolution, not the fix. A 940-line operator guide replaced by a one-line placeholder is a docs regression that will fail human review even if CI is green.

8. GitHub CLI you will actually use

gh is not Git. It talks to GitHub’s API with your logged-in account. On the Mac, install with `brew install gh` and keep SIP/Gatekeeper on. On Windows, GitHub CLI ships with winget or the official MSI.

Command	What it does
<code>gh auth status</code>	Whether this machine can push and open PRs.
<code>gh auth login</code>	Browser or token login. Prefer HTTPS + credential manager.
<code>gh pr view 1168</code>	Title, mergeable state, URL.
<code>gh pr checkout 1168</code>	Fetch and switch to that PR’s branch.
<code>gh pr diff 1168</code>	The PR diff.
<code>gh pr checks 1168</code>	CI status on the head SHA.
<code>gh pr comment 1168 --body "..."</code>	Add an issue comment on the PR.

Command	What it does
<code>gh pr create --fill</code>	Open a PR. Still paste the CyClaw template body.
<code>scripts/check-pr-template.sh</code>	Local body-section gate before <code>gh pr create</code> .

GitHub itself: dirty means “this branch cannot be merged until conflicts are resolved on the branch.” Unstable means “mergeable, but required checks are failing or pending” — that was #1170’s CI state, not a merge conflict. Do not confuse the two.

9. Daily loop, anti-patterns, recovery

Daily loop

```
status → add -p → diff --staged → commit → fetch → merge or rebase → push
```

Before you leave a CyClaw branch: `git diff --check`, confirm the subject has a template prefix, and run the focused tests for the files you touched. Full suite is `GROK_API_KEY=dummy pytest tests/ -q --tb=short`.

Anti-patterns

Don't	Do this instead
<code>git add .</code> on a mixed tree	<code>git add -p</code> , or name the files.
<code>reset --hard</code> a pushed PR commit	<code>git revert --no-edit <sha></code> .
<code>git push --force</code> after a rebase surprise	<code>git push --force-with-lease</code> . Fetch first.
Commit <code>.env</code> or a key in a plist	Keychain / CredMan / mode-600 dotenv outside the repo.
Free-form PR body on CyClaw	Fill <code>.github/PULL_REQUEST_TEMPLATE.md</code> completely.
Resolve docs conflicts by picking one side	Keep both sections when they are additive, as in #1168.
Use Contents-API / <code>push_files</code> for a 40 KB guide	Edit locally, commit, push. Large-file stubs are how 182b8358 happened.
Assume sandbox <code>/tmp/CyClaw</code> still exists	Work in <code>%USERPROFILE%\CyClaw\repo</code> or <code>~/CyClaw/repo</code> .
Disable SIP/Gatekeeper to “fix git”	Install Git via brew or Xcode CLT. SIP stays on.
Amend after someone else pulled your branch	New commit. History is shared now.

Windows notes

Paths with spaces: `git add -- "setup-guide.md"`. `git log` uses a pager; quit with `q`. If `git` is not found, Git for Windows is not on PATH. The credential popup is Git Credential Manager talking to GitHub — sign in once.

macOS notes

`zsh` is the login + interactive shell. Put Homebrew on PATH from `~/.zprofile` with `eval "$(/opt/homebrew/bin/brew shellenv)"`. Do not `sudo brew`. Background Git tasks (fetch mirrors, nothing that writes secrets) belong in a LaunchAgent, not cron. Confirm file `$(command -v git)` says `arm64`.

10. One-page cheat sheet

Job	Command
Where am I?	<code>git status -sb</code>
What changed?	<code>git diff / git diff --staged</code>
Save a snapshot	<code>git add -p → git commit -m "[fix] - ..."</code>
Download	<code>git fetch origin</code>
Upload	<code>git push -u origin HEAD</code>
New branch	<code>git switch -c grok/<kebab></code>
Update from main	<code>git merge origin/main or git rebase origin/main</code>
Undo a shared commit	<code>git revert --no-edit <sha></code>
Undo a local commit	<code>git reset --soft HEAD~1</code>
Panic map	<code>git reflog</code>
PR-shaped diff	<code>git diff origin/main...HEAD</code>
CI / merge state	<code>gh pr view 1168 / gh pr checks 1168</code>
Enter Windows repo	<code>cd \$env:USERPROFILE\.CyClaw\repo</code>
Enter Mac repo	<code>cd ~/.CyClaw/repo</code>

If you only remember four things

1. status before everything. 2. revert, not reset, on a pushed CyClaw PR. 3. keep both additive docs sections. 4. the real repo is under .CyClaw/repo — home is config, repo is git.

Built for C.G. from the 27 Aug 2026 working thread on cgfixit/CyClaw PRs #1168 and #1170. General Git first; CyClaw where it changes the command. Not a repository file — print or keep next to the laptop.